FairPrice — Development Document (PayPal AI Hackathon 2026)

Nov 10, 2026 · @Vi

1. Product overview

FairPrice lets sellers of digital products charge an AI-recommended fair price in every country, and PayPal confirms the buyer's account country before any money is captured. Sellers are protected from VPN abuse; buyers are never overcharged.

Problem. Courses, ebooks, templates and software are usually priced for wealthy countries. Buyers in lower-income countries are priced out, so sellers lose sales they could make at a fair price. Sellers who offer regional discounts face VPN abuse and manual price setup.

Users.

- **Seller:** an independent creator or small software company selling a digital product online.
- **Buyer:** anyone, anywhere, especially in markets where the standard price is unaffordable.

What exists today, and what FairPrice adds.

Product	Country pricing	Who sets prices	Country check at payment
Gumroad	Automatic, 20% to 60% discounts from World Bank data	Gumroad; seller sets only a maximum	Card country and IP
Paddle	Country-specific prices	Seller, manually	Geolocation; buyer can change country
ParityDeals	Discounts by country group	Seller, per group	IP and VPN blocking
Stripe	Local currency only	Not applicable	None
FairPrice	**Per country, per product**	**AI proposes, seller approves**	**PayPal account country**

FairPrice's two genuine differences: AI that prices each product by who actually buys it, and a fair price unlocked by paying with PayPal, which gives priced-out buyers a reason to join PayPal.

Hackathon success criteria.

- [ ] A real sandbox purchase at the correct price for buyers in three countries
- [ ] A VPN buyer is voided with no charge and shown the correct price
- [ ] A traveler is automatically charged their country's lower fair price
- [ ] The AI table beats a pure formula on five benchmark products
- [ ] Complete seller and buyer journeys, hosted on a public link
- [ ] Demo video under 3 minutes

2. Scope

The MVP is one complete loop: AI-priced table, approved by a seller, enforced at a real PayPal sandbox checkout. Everything else waits until that loop works end to end.

Priority	Feature	Notes
Must	Seller setup: product link or description, base price, minimum price, maximum discount	One product per seller is enough for the demo
Must	AI pricing engine with code validator	Grounded in World Bank data; returns price, discount, reason, relevance per country
Must	Editable, approvable price table (AG Grid)	Every approval creates a new, hashed table version
Must	Buyer product page with IP-based fair price and PayPal button	Shows "Fair price for your country, verified by PayPal"
Must	Authorize, check payer country, capture or void	Server-owned amounts; never trust the browser
Must	Result screens for paid, VPN, traveler and unclear cases	Friendly, non-accusatory copy
Must	Orders dashboard (AG Grid)	Shows each decision with its PayPal IDs
Must	Formula-versus-AI benchmark on five products	The innovation moment in the video
Should	Webhook signature verification	Approval and capture events
Should	Log in with PayPal pre-check	Only if the Day 0 test returns the account country
Could	Country preview for sellers	"See this page as a buyer in India"
Could	AI results coach and abuse flags	Uses clearly labelled sample data

Out of scope: onboarding real third-party sellers (needs PayPal partner approval), physical goods and shipping, subscriptions, card-only checkout pricing, multiple currencies beyond USD.

3. User journeys

The seller journey takes about two minutes; the buyer journey is a normal PayPal checkout with one invisible check before capture.

Seller journey

1. Sign in and create a product: paste the product page link or a description, then enter the base price (USD), minimum acceptable price and maximum discount.
2. Choose a goal: "maximize revenue" or "maximize reach."
3. Click **Generate fair prices**. The AI returns a table: price, discount, one-line reason and audience relevance for each country.
4. Review the table in AG Grid. Edit any price; edits are re-checked by the validator instantly.
5. Optionally open the side-by-side view: the formula's table next to the AI's.
6. Click **Approve**. The table is saved as a new version with a hash.
7. Copy the one-line embed snippet or share the hosted product page link.
8. Watch the orders dashboard: each order shows the shown price, payer country, final price and decision.

Buyer journey

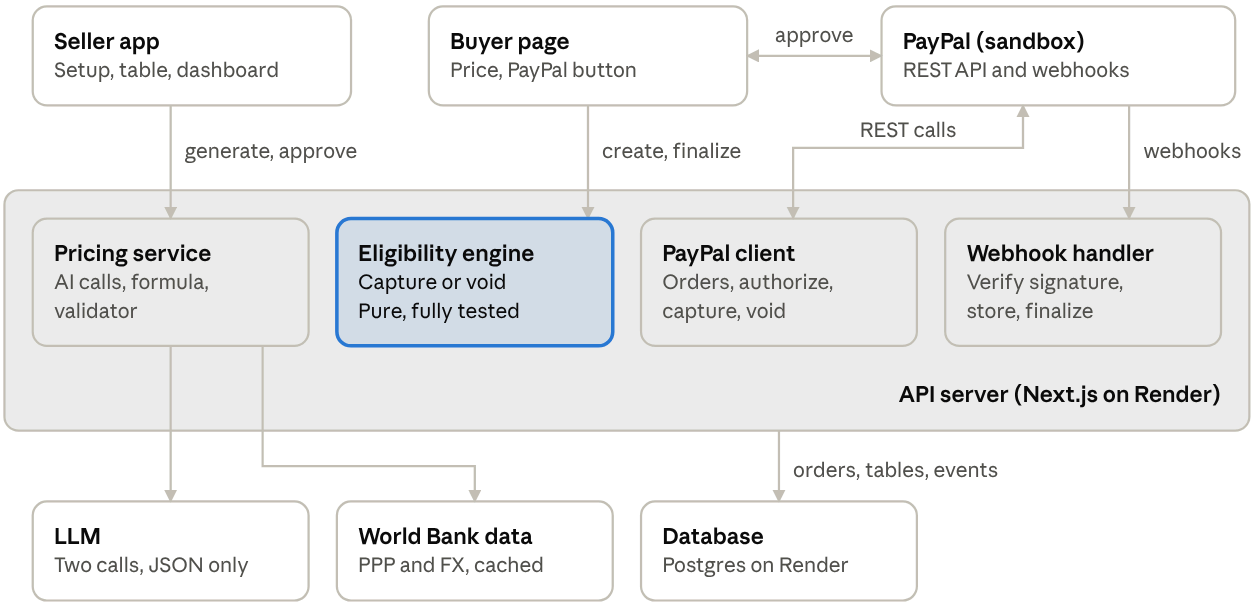
1. Open the product page. The server detects the country from the IP address and shows that country's approved price with a "Verified by PayPal" note.
2. Click the PayPal button and approve the payment in the PayPal window.
3. The server reads the payer's PayPal account country and applies the eligibility rules (section 6).
4. The buyer sees one of four result screens:
    a. **Paid:** "You paid $35, the fair price for India."
    b. **Traveler:** "You were charged $35, the fair price for your country, not $99."
    c. **Country mismatch:** "Your PayPal account is registered in the United States. The price there is $99. You have not been charged." with **Pay $99** and **Cancel**.
    d. **Unclear:** "We couldn't confirm your country, so the standard price applies. You have not been charged." with **Pay standard price**.

The prices in these screens are demo examples; real values come from the approved table.

4. System architecture

FairPrice is one web app with a server that owns every price and every PayPal call; the browser only displays prices and opens the PayPal window.

The server owns every price and every money decision



system architecture · 3 clients and services, 4 server parts, 3 back-end stores

The browser never sends an amount: prices come from the pricing service's approved table, and only the eligibility engine decides whether PayPal captures or voids.

Component	Responsibility	Suggested technology
Seller app	Product setup, AI table review and approval, embed snippet, orders dashboard	Next.js (React), AG Grid
Buyer page	Shows the fair price, PayPal button, result screens	Next.js page, PayPal JavaScript SDK
API server	Server-owned prices, PayPal calls, webhooks, audit log	Next.js API routes or Node.js with Express
Eligibility engine	Pure function deciding capture, capture lower, or void	TypeScript module with unit tests
AI pricing engine	Proposes per-country prices with reasons	Any LLM with structured JSON output
Validator	Enforces floors, caps and consistency on AI output	TypeScript module with unit tests

Component	Responsibility	Suggested technology
Geo lookup	Country from IP for the displayed price	An IP geolocation library or service
Data	Purchasing-power and exchange-rate data	World Bank API, cached locally
Database	Products, price tables, orders, events	Postgres (or SQLite for speed)
Hosting	Public demo link	Render

Request flow at checkout. The buyer page asks the server to create an order; the server picks the price from the approved table and calls PayPal. After the buyer approves, the server reads the order, runs the eligibility engine, then captures or voids. Webhooks confirm the final state independently.

5. PayPal integration

Every order uses authorize intent: the buyer approves, the server authorizes, reads the payer's account country, then captures the eligible amount or voids. Capturing less than the authorized amount is what lets travelers pay their own country's lower price.

Sandbox setup (Tuesday, first two hours).

- One REST app in the PayPal Developer Dashboard with sandbox client ID and secret.
- One sandbox **business** account located in the **US**. A business account in India cannot accept payments from buyers in India, because PayPal doesn't process domestic India payments.
- Three sandbox **personal** buyer accounts: India, Brazil, United States. Add a fourth (Nigeria or Indonesia) if time allows.
- Webhook listener registered for the events below, pointing to the deployed Render URL.

Calls, in order.

Step	Call	Key details
1	`POST /v1/oauth2/token`	Server-side only; cache the token until it expires

Step	Call	Key details
2	`POST /v2/checkout/orders`	`intent: AUTHORIZE`; amount from the approved table; `custom_id` = product, table version, shown country; `shipping_preference: NO_SHIPPING` (digital goods)
3	JavaScript SDK button	`createOrder` calls our server; `onApprove` calls our server; the browser never sends an amount
4	`POST /v2/checkout/orders/{id}/authorize`	`PayPal-Request-Id` header for idempotency; `Prefer: return=representation`
5	Read payer country	`payer.address.country_code`, falling back to `payment_source.paypal.address.country_code`; or `GET /v2/checkout/orders/{id}`
6a	`POST /v2/payments/authorizations/{id}/capture`	Amount = eligible price (equal or lower); `final_capture: true`; unique `invoice_id`; idempotency header
6b	`POST /v2/payments/authorizations/{id}/void`	Country mismatch or unclear; buyer is never charged
7	`POST /v1/notifications/verify-webhook-signature`	Verify every webhook before acting on it

Reference: Authorize payment for order and Show order details. Confirm exact field paths in the Day 0 test; the fallback order in step 5 is an assumption to check.

Webhooks to subscribe. `PAYMENT.AUTHORIZATION.CREATED`, `PAYMENT.CAPTURE.COMPLETED`, `PAYMENT.AUTHORIZATION.VOIDED`, `PAYMENT.CAPTURE.DENIED`. Store each event ID once; ignore duplicates.

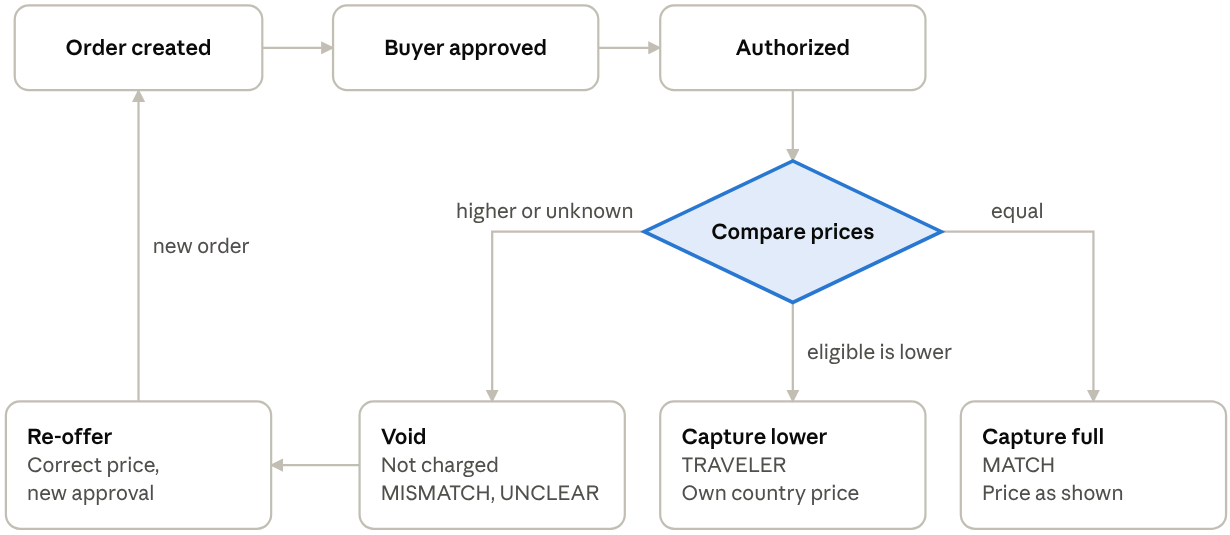
Optional pre-check: Log in with PayPal. If Tuesday's test shows it returns the account country, the buyer page offers "Unlock your fair price with PayPal" before checkout. The pre-capture check in step 5 still runs, because a buyer could log in with one account and pay with another.

Timing. Capture or void happens within seconds of approval, well inside PayPal's authorization windows; no long-lived holds are needed.

6. Eligibility engine

The eligibility engine is a pure function with no AI and no network calls: given the authorized amount, the payer's country and the approved table, it returns capture, capture lower, or void. It is the most tested code in the project.

No money is captured until the payer country is checked



order states · 1 decision, 3 outcomes, 1 loop back

Every order passes one decision: the eligible price for the payer's country is compared with the authorized amount, and only an equal or lower price is ever captured.

```
type Decision =
  | { action: "CAPTURE"; amountCents: number; reason: "MATCH" | "TRAVELER" }
  | { action: "VOID"; reason: "MISMATCH" | "UNCLEAR" | "NOT_PAYPAL";
offerCents: number };

function decide(input: {
  authorizedCents: number;          // what the buyer approved
  payerCountry: string | null;      // from the PayPal order
  paidWithPayPalAccount: boolean;   // false for guest card checkout
  table: PriceTable;                // the version stored in custom_id
  standardCents: number;            // the base price
  requirePayPalForFairPrice: boolean;
}): Decision
```

Rules, applied in order.

1. Guest card payment and the seller requires PayPal for fair prices: void, offer the standard price (`NOT_PAYPAL`).
2. No payer country: void, offer the standard price (`UNCLEAR`).
3. Eligible price = the table's price for the payer country, or the standard price if that country isn't in the table.
4. Eligible equals authorized: capture the full amount (`MATCH`).
5. Eligible is lower than authorized: capture the eligible amount (`TRAVELER`).
6. Eligible is higher than authorized: void, offer the eligible price (`MISMATCH`).

All amounts are compared as integer cents. The table version comes from the order's `custom_id`, so a buyer always gets the prices that were live when they clicked, even if the seller re-approves mid-checkout.

Edge cases.

Situation	Handling
Buyer double-clicks or refreshes after approving	Same `PayPal-Request-Id` per order step; repeated calls return the first result
Webhook arrives before the approve call finishes	Both paths call one idempotent `finalizeOrder(orderId)`; the database row lock decides who runs it
Authorization declined	Show "Payment wasn't approved" with a retry button; no decision runs
Capture call fails	Retry with the same request ID up to 3 times, then mark the order "Needs review"
Payer country not in the table	Standard price applies (rule 3)
Seller re-approves the table during checkout	The order keeps its original table version

Unit test matrix (minimum).

Test	Authorized	Payer country	Table price there	Expected
India buyer	$35	IN	$35	Capture $35, MATCH
VPN user	$35	US	$99	Void, offer $99
Traveler	$99	IN	$35	Capture $35, TRAVELER
Unknown country	$35	none	not applicable	Void, offer $99
Country missing from table	$35	FR	none (standard $99)	Void, offer $99
Guest card, PayPal required	$35	none	not applicable	Void, offer $99
Cents precision	$34.99	IN	$34.99	Capture $34.99
Same country, table changed later	$35 (table v3)	IN	$30 in v4	Capture $35 from v3

Amounts in the matrix assume a $99 standard price and an illustrative India price.

7. AI pricing engine

The AI starts from the same World Bank data a formula uses, then adjusts for who actually buys this product, payment fees, resale risk and local price endings. A code validator has the final say on every number.

Inputs.

Input	Source
Product page text or description	Seller; fetched and cleaned server-side
Base price (USD), minimum acceptable price, maximum discount, goal (revenue or reach)	Seller form

Input	Source
Price-level ratio per country	World Bank: PPP conversion factor (`PA.NUS.PPP`) divided by official exchange rate (`PA.NUS.FCRF`), latest year available, cached as JSON
Fee schedule	Config file for the merchant's PayPal country; confirm current rates on the PayPal fees page
Target countries	About 40 large internet markets for the demo; the formula covers all others

Baseline formula (what the AI must beat). With the US as base, a country's multiplier is:

```
M_c = 1 + s \left( \frac{\mathrm{PPP}_c}{\mathrm{FX}_c} - 1 \right), \quad 0 \le s \le 1
```

The formula price is base × M, clamped to the seller's minimum and maximum discount. It treats every product the same way.

Two AI calls, not one.

1. **Audience analysis.** From the product page, return a profile: category, language, individual or business buyer, how easily it can be shared or resold, and relevance (high, medium, low) by region with a one-line reason. This step is shown on screen, which makes the AI's thinking visible to judges.
2. **Pricing.** Given the profile, ratios, fees and seller constraints, return the country table.

What the AI reasons about that the formula can't.

- **Audience relevance:** deep discounts where real buyers exist; little or none where they don't.
- **Fee floor:** a price whose fees eat most of the sale is raised or flagged.
- **Resale and sharing risk:** easily shared files get shallower discounts.
- **Price endings:** round to natural local-feeling USD points (for example $29 rather than $28.63).
- **Goal:** "reach" accepts deeper discounts; "revenue" protects margin.

Output schema (pricing call).

```
{
  "productSummary": "string, max 25 words",
  "rows": [
    {
      "country": "IN",
      "priceUsd": 35,
      "discountPct": 65,
      "relevance": "high",
      "reason": "Large English-speaking developer audience; deep discount
drives volume.",
      "flags": []
    }
  ]
}
```

Prompting rules. Low temperature; JSON-only output against the schema; reasons under 20 words; data passed as JSON, never paraphrased; the model is told the bounds but the validator enforces them.

Validator (plain code).

- Floor = the higher of the seller's minimum and the fee floor; price never below it.
- Price never above the base price; discount never above the seller's maximum.
- Every target country present exactly once; prices rounded to whole cents.
- Warn when a lower-income country is priced above a higher-income one and the reason doesn't explain it.
- On failure: re-ask once with the errors listed; rows still failing fall back to the formula and are flagged in the grid.

Benchmark (the innovation moment). Five products: a coding course, a US tax-preparation guide, a design template pack, an English-language ebook, a software license. For each, show formula and AI tables side by side, plus two counts: rows priced below the fee floor, and discounts given in low-relevance countries. No conversion or revenue uplift is claimed without real sales data.

8. Data model

Five tables: a product has many price-table versions; each order points at the exact version it used; every PayPal call and webhook is logged.

```
CREATE TABLE products (
  id              TEXT PRIMARY KEY,
  seller_id       TEXT NOT NULL,
  title           TEXT NOT NULL,
  source_url      TEXT,
  description     TEXT,
  base_cents      INTEGER NOT NULL,
  min_cents       INTEGER NOT NULL,
  max_discount_pct INTEGER NOT NULL,
  goal            TEXT CHECK (goal IN ('revenue','reach')),
  require_paypal  BOOLEAN DEFAULT TRUE,
  audience_profile JSON,                -- output of AI call 1
  created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE price_tables (
  id              TEXT PRIMARY KEY,
  product_id      TEXT REFERENCES products(id),
  version         INTEGER NOT NULL,
  status          TEXT CHECK (status IN ('draft','approved','superseded')),
  rows            JSON NOT NULL,          -- country, price, discount,
relevance, reason, flags
  formula_rows    JSON NOT NULL,          -- baseline for the side-by-side
view
  content_hash    TEXT NOT NULL,          -- sha256 of rows; shown on orders
  approved_at     TIMESTAMP,
  UNIQUE (product_id, version)
);

CREATE TABLE orders (
  id               TEXT PRIMARY KEY,
  product_id       TEXT REFERENCES products(id),
  price_table_id   TEXT REFERENCES price_tables(id),
  paypal_order_id  TEXT UNIQUE,
  authorization_id TEXT,
  capture_id       TEXT,
  shown_country    TEXT,                  -- from IP
  shown_cents      INTEGER NOT NULL,      -- authorized amount
  payer_country    TEXT,                  -- from PayPal
  eligible_cents   INTEGER,
  decision         TEXT,                  -- MATCH, TRAVELER, MISMATCH,
UNCLEAR, NOT_PAYPAL
  status           TEXT NOT NULL,         -- see state diagram
```

```
  created_at       TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at       TIMESTAMP
);

CREATE TABLE webhook_events (
  event_id     TEXT PRIMARY KEY,          -- PayPal event ID; duplicates
ignored
  event_type   TEXT NOT NULL,
  resource_id  TEXT,
  verified     BOOLEAN NOT NULL,
  payload      JSON NOT NULL,
  received_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE audit_log (
  id         INTEGER PRIMARY KEY AUTOINCREMENT,
  order_id   TEXT,
  step       TEXT NOT NULL,               -- create, authorize, decide,
capture, void
  request_id TEXT,                        -- PayPal-Request-Id used
  outcome    TEXT NOT NULL,
  detail     JSON,
  at         TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

The dashboard reads `orders` joined to `price_tables`, so every row shows which approved table version priced it.

9. Backend API

Twelve endpoints; no buyer-facing endpoint accepts an amount, so prices can only come from the server's approved table.

Method	Path	Purpose	Returns
POST	`/api/products`	Create product with base price, minimum, max discount, goal, PayPal requirement	`productId`
POST	`/api/products/:id/analyze`	AI call 1: audience profile	`audienceProfile`

Method	Path	Purpose	Returns
POST	`/api/products/:id/price-tables`	AI call 2, validator, formula baseline	`priceTableId`, `version`, `rows`, `formulaRows`, `warnings`
PATCH	`/api/price-tables/:id/rows/:country`	Seller edits one price; validator re-runs	Updated row, or 422 with reasons
POST	`/api/price-tables/:id/approve`	Lock the version and hash it	`version`, `contentHash`
GET	`/api/products/:id/orders`	Orders for the dashboard	Order rows with decisions and PayPal IDs
GET	`/api/benchmark`	Five-product formula versus AI comparison	Side-by-side tables and counts
GET	`/api/quote?productId=`	Price for the visitor's IP country	`shownCountry`, `priceCents`, `standardCents`
POST	`/api/checkout/orders`	Create the PayPal order (body: `productId` only)	`paypalOrderId`
POST	`/api/checkout/orders/:paypalOrderId/finalize`	Authorize, read country, decide, capture or void	Result object (below)
POST	`/api/checkout/reoffer`	New order at the offered price after a void (body: `previousOrderId`)	`paypalOrderId`
POST	`/api/webhooks/paypal`	Verify, store, call the same idempotent finalize	200

Finalize response examples.

```
{ "result": "PAID", "chargedCents": 3500, "country": "IN",
  "message": "You paid $35, the fair price for India." }

{ "result": "TRAVELER", "chargedCents": 3500, "country": "IN",
  "message": "You were charged $35, the fair price for your country." }

{ "result": "MISMATCH", "chargedCents": 0, "country": "US", "offerCents":
9900,
  "message": "Your PayPal account is registered in the United States. The
price there is $99. You have not been charged." }
```

Rules for every endpoint. Seller endpoints require a session. Errors return `{ code, message }` with no stack traces. The re-offer endpoint reads the offer from the voided order row, never from the request body.

10. Frontend and UX

Eight screens: six for the seller, two for the buyer. The buyer side must feel like a normal, trustworthy PayPal checkout; the seller side must make the AI's reasoning visible.

#	Screen	Who	Key elements
1	Product setup	Seller	Link or description, base price, minimum price, max discount, goal toggle, "Require PayPal for fair prices" switch
2	Audience analysis	Seller	AI profile as chips: category, buyer type, shareability; region relevance with one-line reasons
3	Price table	Seller	AG Grid: country, relevance, formula price, AI price (editable), discount, net after fees, reason, flags; **Approve version** button
4	Formula versus AI	Seller	Two columns side by side; rows where they differ most highlighted with the AI's reason
5	Publish	Seller	One-line embed snippet, hosted page link, approved version and hash

#	Screen	Who	Key elements
6	Orders dashboard	Seller	AG Grid: time, shown country, payer country, shown price, charged, decision, PayPal IDs, table version
7	Product page	Buyer	Product title, fair price in large type, "Fair price for India, verified by PayPal at checkout," PayPal button
8	Result	Buyer	One of the four states in section 3, each saying clearly whether money moved

AG Grid usage. Use Community features only: editable cells with validator feedback (invalid cells turn red with the reason on hover), custom cell renderers for relevance and decision chips, sorting and filtering by relevance or decision, and row highlighting for flagged rows. Both grids should look deliberate, not like default spreadsheets.

Copy rules.

- Always state whether money moved: "You have not been charged" or "You paid $35."
- Never accuse: say "Your PayPal account is registered in the United States," not "VPN detected."
- Explain prices in one sentence, never with formulas.

Design rules.

- One accent colour; neutral surfaces; light and dark modes.
- Buyer page designed mobile-first; it will be shown on a phone in the video.
- Use the official PayPal SDK button and don't restyle it.
- Loading states for both AI calls, with a short label of what the AI is doing.

11. Security, reliability and privacy

The server owns every amount, every PayPal call is idempotent and logged, and the AI can never move money.

Risk	Control
Buyer edits the price in the browser	No buyer endpoint accepts an amount; the server reads the approved table

Risk	Control
Duplicate charges from retries	`PayPal-Request-Id` on authorize, capture and void; one `finalizeOrder` path with a database lock
Forged webhooks	Verify every event with PayPal's verify-signature call before acting; store event IDs once
AI produces a bad price	Validator enforces floors and caps; the seller must approve; AI output never reaches checkout unapproved
Prompt injection from a product page	Page text passed as data inside delimiters; the model has no tools and can only return JSON
Leaked secrets	PayPal secret and AI key in environment variables only; never in the repo or the browser; `.env.example` with blanks
Lost audit trail	Every PayPal call and decision written to `audit_log` with request ID and outcome
Personal data	Store payer country and PayPal IDs only; no names, emails or addresses in the database or AI prompts

Honest claims. The product and video say the account country is "strong evidence," never "proof of residence," and never claim real revenue or user growth from sandbox data.

12. Testing strategy

Four layers, cheapest first; the end-to-end sandbox run on Wednesday night is the gate for feature freeze.

Layer	What it covers	Tooling	Owner
Unit	Eligibility engine (section 6 matrix), validator rules, formula math, cents rounding	Vitest or Jest	Backend, AI
Contract	AI output always parses against the JSON schema; 20 runs per benchmark product	Schema validator (Zod or Ajv)	AI

Layer	What it covers	Tooling	Owner
Integration	Real sandbox: create, approve, authorize, capture lower, void, webhook verify	Script with sandbox buyer logins	Backend
End-to-end	Full seller and buyer journeys in a browser, all four result states	Playwright, plus a manual run	QA

Demo rehearsal checklist.

- [ ] Fresh sandbox buyers for India, Brazil and the US log in without extra prompts
- [ ] India buyer pays the fair price; capture ID appears in the dashboard
- [ ] US buyer browsing with a VPN set to India is voided and re-offered the US price
- [ ] India buyer browsing from a US IP is captured at the India price
- [ ] Benchmark view loads instantly (precomputed)
- [ ] Hosted Render app wakes up within the video's first seconds (ping it before recording)

13. Environments and deployment

One public repository, one Render web service, sandbox credentials only; a judge must be able to run it locally or open the hosted link without help.

Repository layout.

```
fairprice/
  app/              Next.js pages: seller app, buyer page, result screens
  app/api/          API routes (section 9)
  lib/paypal/       token, orders, authorize, capture, void, webhook verify
  lib/eligibility/  decide() + tests
  lib/pricing/      World Bank loader, formula, AI calls, validator + tests
  data/             cached World Bank JSON, fee config, benchmark products
  db/               schema.sql, seed script
  tests/            integration and Playwright tests
  .env.example
  LICENSE
  README.md
```

Environment variables.

Variable	Purpose
`PAYPAL_CLIENT_ID`	Sandbox REST app client ID (also used by the browser SDK)
`PAYPAL_CLIENT_SECRET`	Sandbox secret; server only
`PAYPAL_WEBHOOK_ID`	Needed to verify webhook signatures
`PAYPAL_API_BASE`	`https://api-m.sandbox.paypal.com`
`AI_API_KEY`, `AI_MODEL`	The LLM used for both AI calls
`DATABASE_URL`	Postgres on Render, or a SQLite file locally
`SESSION_SECRET`	Seller sessions

Render deployment. One web service from the main branch, a Render Postgres instance, environment variables set in the dashboard, and the PayPal webhook pointed at `/api/webhooks/paypal` on the Render URL. Seed one demo seller and the five benchmark products on deploy.

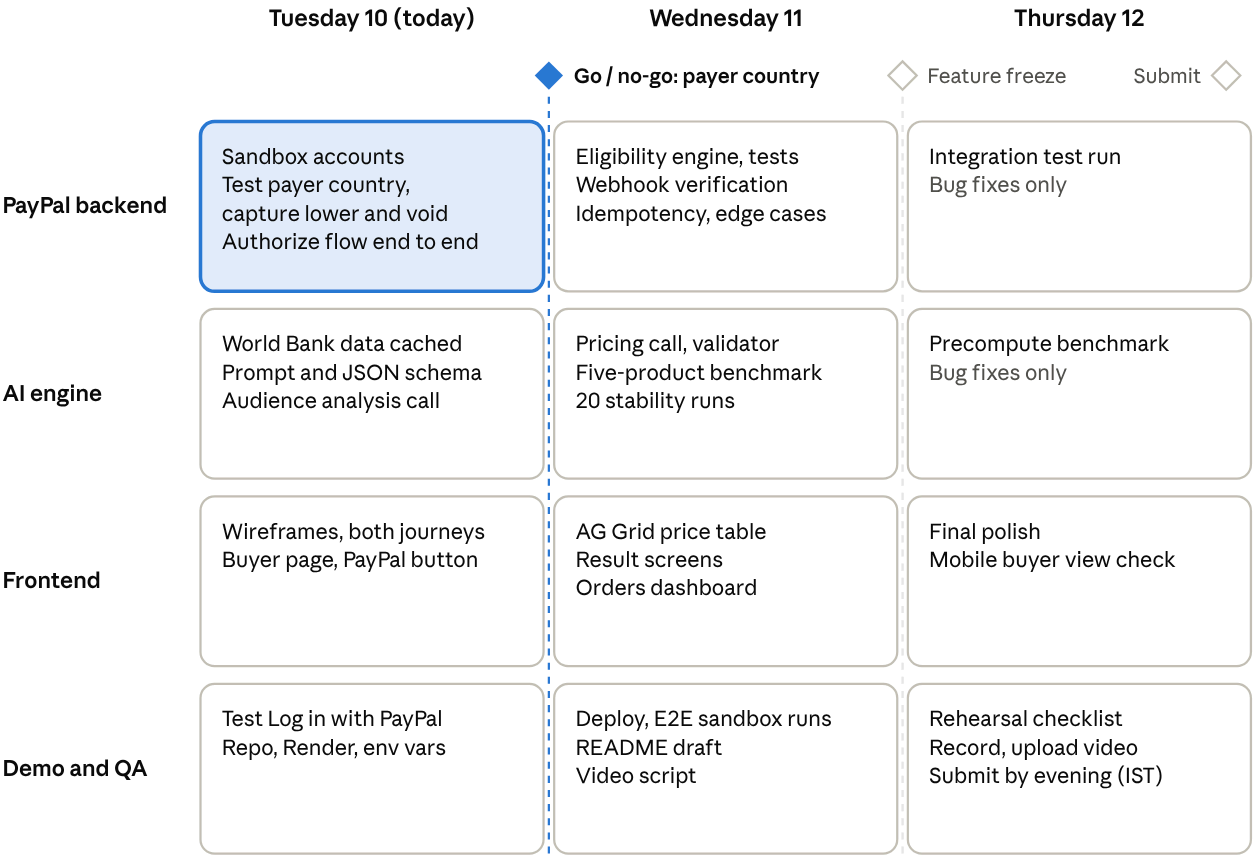
README must include. What FairPrice does in three sentences; the hosted link; local setup in five commands or fewer; sandbox test buyer instructions; which AI tools are used and how; the license. These match the hackathon's submission requirements.

14. Team roles and timeline

Today is Tuesday 10 November, so the plan runs Tuesday to Thursday: test the premise first, build the full loop by Wednesday night, and spend Thursday on quality and the video.

The payer-country test on Tuesday decides everything else

Team plan by workstream, 10 to 12 November 2026 (India time)



team plan · 4 workstreams, 3 days, 3 gates

Nothing else starts until the payer-country test passes; after the Wednesday-night freeze, only bug fixes go in.

Workstream	Owns	Hands off to
PayPal backend	Sections 5, 6, 8, 9, 11	Frontend: working checkout endpoints by Tuesday night
AI engine	Section 7 and the benchmark	Frontend: table JSON by Wednesday noon
Frontend	Section 10	QA: complete journeys by Wednesday evening
Demo and QA	Sections 12, 13, 15, 16	Everyone: rehearsal results Thursday morning

Daily check-ins. 15 minutes at the start of each day and before the freeze: what's done, what's blocked, what gets cut.

15. Demo video script

One story in 2 minutes 50 seconds: a creator, a buyer in India, a VPN user and a traveler, all on the real PayPal sandbox.

Time	Scene	On screen	Voiceover
0:00–0:20	The problem	A course page priced at $99; visitors from India and Brazil, almost no sales	"Digital products are priced for rich countries. Most of the world can't pay $99."
0:20–0:40	Seller setup	Paste a product link; the AI audience analysis appears	"FairPrice reads the product and works out who actually buys it."
0:40–1:00	AI versus formula	Side-by-side; the US tax guide stays at full price abroad, the coding course is deeply discounted in India	"A formula discounts everything. The AI discounts where real buyers are."
1:00–1:10	Approve	Seller approves; version and hash appear	"The seller approves. Prices are locked and versioned."
1:10–1:40	Fair purchase	Phone view: India buyer sees $35, pays with PayPal sandbox; success screen	"A buyer in India pays a fair price with PayPal."
1:40–2:05	VPN caught	US buyer with an Indian IP approves $35; result: not charged, offered $99	"PayPal confirms the account country before any money moves. Nobody is charged a wrong amount."
2:05–2:20	Traveler	India buyer on a US IP approves $99, is charged $35	"It works both ways: buyers are never overcharged either."
2:20–2:35	Trust	Orders dashboard: decisions, PayPal capture IDs, table version	"Every decision is recorded with its PayPal ID."

Time	Scene	On screen	Voiceover
2:35–2:50	Vision	Title card	"PayPal's verified accounts can become the trust layer for fair global pricing, and a reason for millions of priced-out buyers to join PayPal."

Recording tips. Record each scene separately and edit together; use a real phone frame for the buyer scenes; show the PayPal sandbox window, not a mock. Prices shown are demo values from the approved table.

16. Submission checklist

The deadline is Thursday, 12 November 2026 at 12:00 PM Pacific (1:30 AM Friday, India time); submit by Thursday evening India time to leave a buffer. Requirements are from the hackathon page.

- [ ] Project uses the PayPal developer platform (sandbox) and AI meaningfully
- [ ] Text description of features and functionality on Devpost
- [ ] Working demo: hosted Render link, plus clear run instructions in the README
- [ ] Description of which AI tools are used and how
- [ ] Public GitHub repository with all source code, assets and instructions
- [ ] Open-source license file visible at the top of the repository
- [ ] Demo video under 3 minutes, uploaded to YouTube and set to public
- [ ] Sponsor tools noted in the submission (AG Grid, Render) for sponsor prize eligibility
- [ ] Sandbox test buyer logins included for judges, or a clearly labelled demo mode
- [ ] Final check: open the hosted link in a private window and complete one purchase

17. Risks and fallbacks

The biggest risk is the premise itself: if the sandbox doesn't return the payer's country, the core check fails, so it is tested first, on Tuesday morning, before any other build work.

Risk	Likelihood	Day 0 test	Fallback
Payer country missing from the order	Medium	Authorize as India, Brazil and US buyers; log the full response	Try `payment_source.paypal.address`, then Log in with PayPal; if all fail, rethink the project before Tuesday
Capture below the authorized amount refused	Low	Authorize $99, capture $35	Void and create a new order at the lower price for travelers
Log in with PayPal doesn't return a country	Medium	Request the address scope in sandbox	Skip the pre-check; the pre-capture check alone enforces pricing
`NO_SHIPPING` removes the payer address	Low	Compare responses with and without it	Use the default shipping preference
AI output unstable or slow	Medium	20 runs per benchmark product	Cache approved tables; precompute the benchmark; formula fills failing rows
Render cold start during judging	Medium	Measure wake time	Keep-alive ping; local run instructions in the README
Team runs out of time	Medium	Wednesday night end-to-end run	Cut the Could items in section 2 first, then the Log in pre-check

Decision rule. If Tuesday's test fails and no fallback returns a country by Tuesday evening India time, stop and choose a different project rather than build on an unverified premise.

18. Judging criteria alignment

Innovation is the weakest criterion because regional pricing already exists, so first place depends on near-top scores in the other four, each worth 20%.

Criterion	What judges ask	How FairPrice earns it	Proof in the demo
Technical implementation	Thorough, skilful use of PayPal and AI; working and non-trivial	Authorize, capture lower, void, idempotency, verified webhooks; two-step AI with a code validator	Real sandbox IDs in the dashboard; tests in the repo
Innovation	How novel; how it differs from existing concepts	AI prices each product by its real audience; fair price unlocked by a PayPal account	Formula versus AI side-by-side
Real-world impact	A specific problem for a specific audience, solved in what is shown	Creators lose sales; buyers are priced out; both fixed in one flow	India purchase and fairness-both-ways scenes
Product design	A complete, coherent product, not a proof of concept	Polished seller and buyer journeys, clear result states, purposeful grids	Phone-framed buyer scenes; clean dashboards
Demo and presentation	Clear end-to-end story: problem, audience, why it matters	One story in under 3 minutes	Section 15 script

Prize strategy. Aim for first place; the same build competes for Best Demo Delivery and Most Impactful, and AG Grid and Render use adds sponsor prize eligibility. Confirm in the official rules how many prizes one project can win.